

DESIGN VALIDATION

Simulation

1. Kinematic Validation (Unity Physics Engine)

Initial autonomous testing was conducted in Unity using dual Time-of-Flight (ToF) distance sensors for corridor centering. The vehicle navigates the environment entirely autonomously, powered by custom-built obstacle avoidance logic, an autonomous motion controller, and real-time robot telemetry scripts. The simulation successfully validated the mechanical chassis design, demonstrating that the lowered centre of mass provides a stable turning radius through tight chicanes without tipping or nose-diving at racing speeds.

- **Environment Credit:** The simulation environment utilizes the "Cartoon Race Track - Oval" asset package created by RCC Design via the Unity Asset Store.
- **Video Evidence:** To ensure smooth playback and comply with file size limits, the kinematic simulation video is hosted externally.
- **Watch the Unity Kinematic Simulation:** [Google Drive Link](#)

NAVIGATION ARCHITECTURE

2. Electronic Justification Pivot & Hybrid Navigation

While the chassis proved mechanically stable, the Unity simulation data revealed a critical flaw in purely ToF-based navigation: on track straights wider than 2.5 meters, the sensors lose simultaneous wall contact. Without lane-heading awareness, the vehicle suffers from oscillation and diagonal drifting.

This simulated failure provided the exact data needed to finalize our electronic architecture. To prevent drifting in the physical Robo Grand Prix, we upgraded to a Hybrid Navigation System:

- **Predictive Vision (Raspberry Pi Camera):** A custom OpenCV pipeline looks ahead to extract painted lanes and calculate upcoming corner severity, providing proactive steering angles and physics-based speed caps.
- **Reactive Safety (ToF & IR Sensors):** The STM32 microcontroller simultaneously polls the dual ToF sensors for dynamic obstacle avoidance (e.g., other robots on the track) and an IR array for immediate road-boundary correction.

This sensor fusion ensures that while the camera dictates the racing line, the low-level hardware sensors retain ultimate authority to trigger an emergency brake or avoidance manoeuvre if a collision is imminent.

EMBEDDED CONTROL VALIDATION

3. State-Machine Validation (Wokwi Logic Simulation)

To validate the upgraded control architecture, the core C++ navigation state-machine and I2C sensor parsing logic were simulated using Wokwi. This confirmed that our STM32 microcontroller can successfully integrate real-time telemetry from the IMU, INA219 current sensor, and wheel encoders while executing our PID steering algorithms.

- **Link to Wokwi Simulation Project:** [STM32 Copy - Wokwi ESP32, STM32, Arduino Simulator](#)

(Note on Hardware Annotations: The Wokwi simulation environment does not natively support textual component annotations. However, the E-Stop logic is fully implemented and tested within the C++ code itself. For the visually annotated E-Stop and ON/OFF switches required by the rubric, please reference the master Proteus schematics located in the Electronic_Design folder).